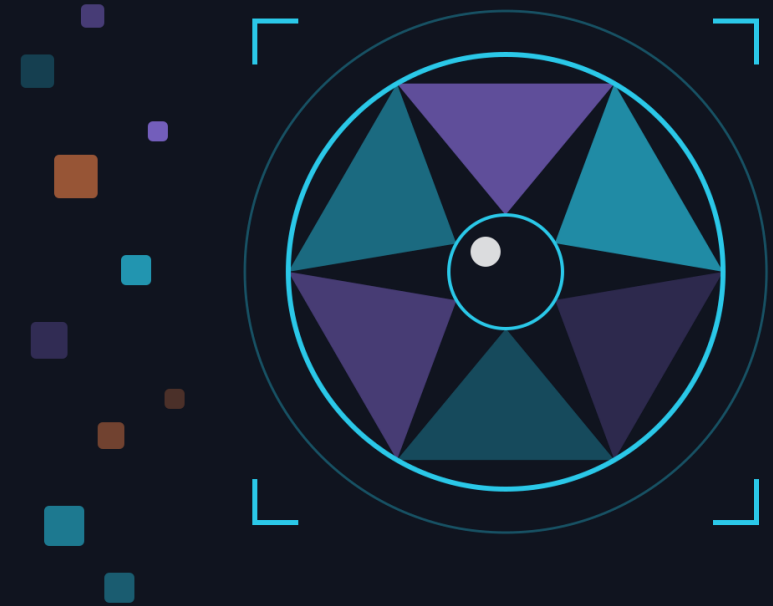


COMPUTER VISION

Introduction to Computer Vision

Teaching machines to see, and to make sense of what they see

Prathyusha Dokku



What is Computer Vision?



Field of AI focused on interpreting visual data



Mimics human vision system



Involves image processing, recognition, and understanding

It never truly “sees” — it reads pixel values and finds patterns in the numbers.

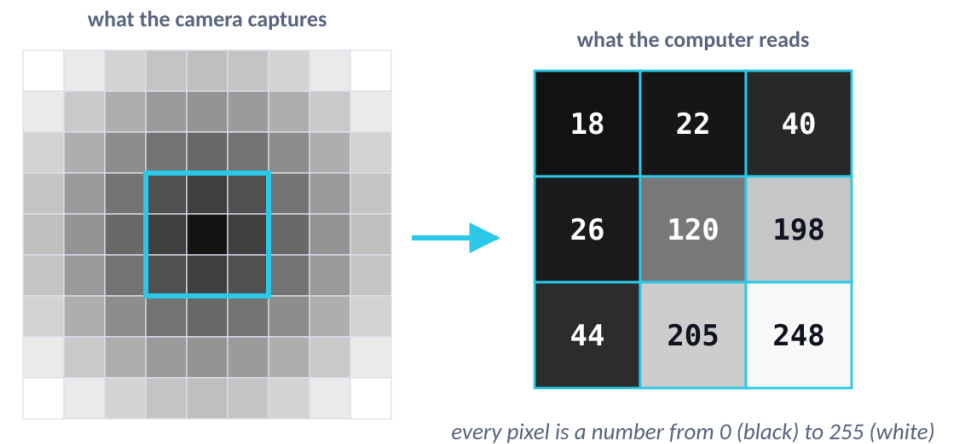


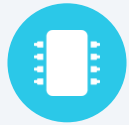
Photo tagging suggestions are this idea in production.

Why is Computer Vision Important?



Visual data = majority of human information

Most of what we take in arrives as images and video



Enables automation and decision-making

Machines can act on what they observe, not just on typed input



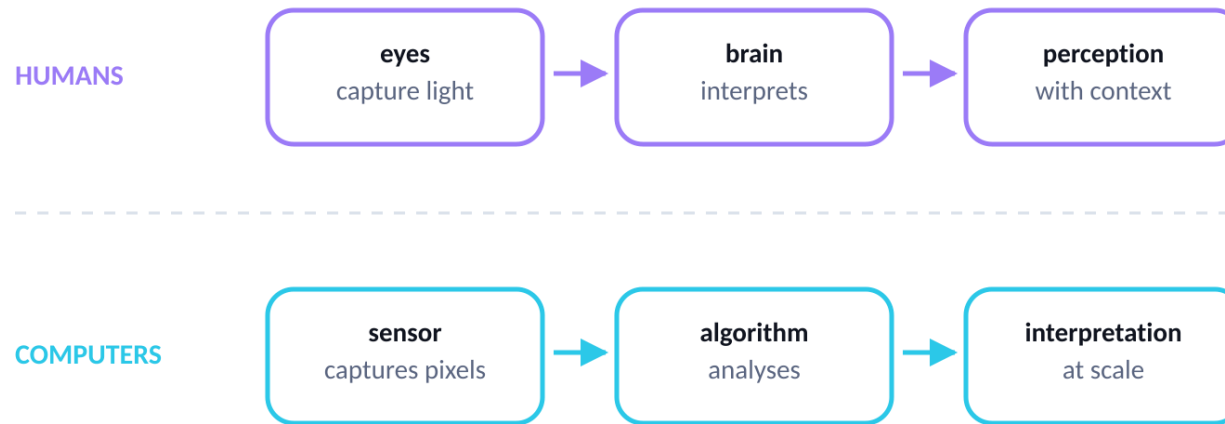
Used in industries from healthcare to transportation

Self-driving cars, X-ray analysis, Face ID

WHY NOW

- 1 Cameras became cheap and are now in everything.
- 2 Storage and GPUs made it affordable to train on millions of images.
- 3 Deep learning removed the need to hand-design what to look for.
- 4 The result: tasks that were research problems in 2012 now ship in phones.

Human Vision vs. Computer Vision



Humans: eyes + brain → perception

Effortless context, tiny sample sizes

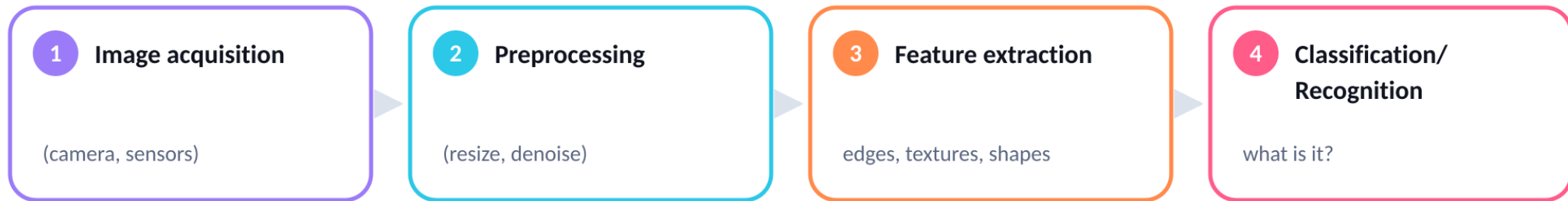
Computers: sensors + algorithms → interpretation

Pixels in, probabilities out

Computers lack context but excel in scale

Never tired, never distracted, millions of frames

The Computer Vision Pipeline



WHAT HAPPENS AT EACH STEP

- 1 Acquire** — a sensor turns light into a grid of numbers.
- 2 Preprocess** — resize, denoise, adjust contrast so every image looks comparable.
- 3 Extract features** — reduce millions of pixels to the patterns that matter.
- 4 Classify** — score those patterns against what the model has learned.

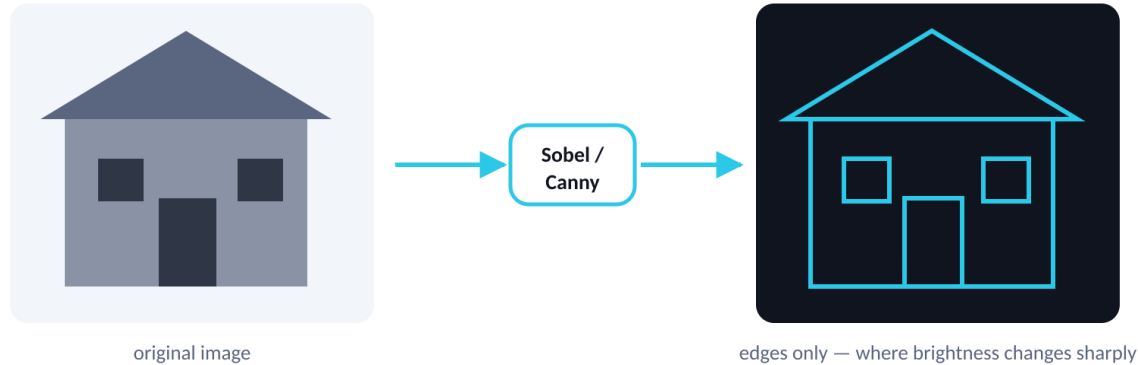
Self-checkout, step by step

The camera captures the apple, enhances the image, extracts its shape and colour, then matches it against known produce.

Garbage in, garbage out

Most accuracy problems trace back to step 2, not the model.

Image Processing Basics



HOW EDGE DETECTION WORKS

- 1 Compare each pixel with its neighbours.
- 2 Where brightness jumps sharply, mark it — that is an edge.
- 3 Keep only the strongest, thinnest lines and discard the rest.



Filtering (blur, sharpen)



Edge detection (Canny, Sobel)



Color transformations (RGB → grayscale)

WHY GRAYSCALE FIRST

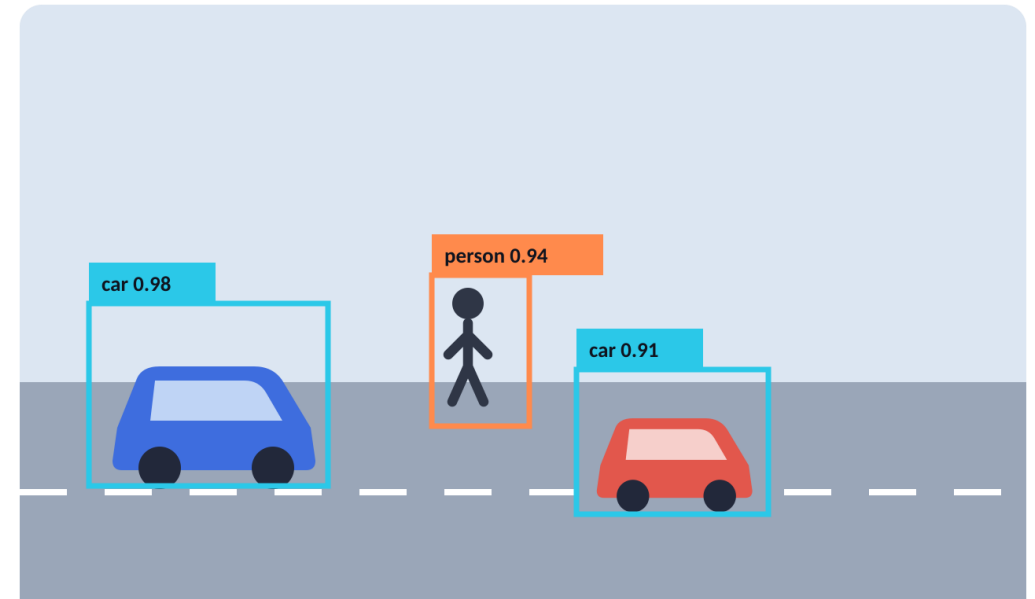
- 1 Colour triples the data for little gain when shape is the signal.
- 2 Edges are brightness changes — one channel is enough to find them.

Object Detection

- Identifying what and where
- Bounding boxes around objects
- Algorithms: YOLO, Faster R-CNN

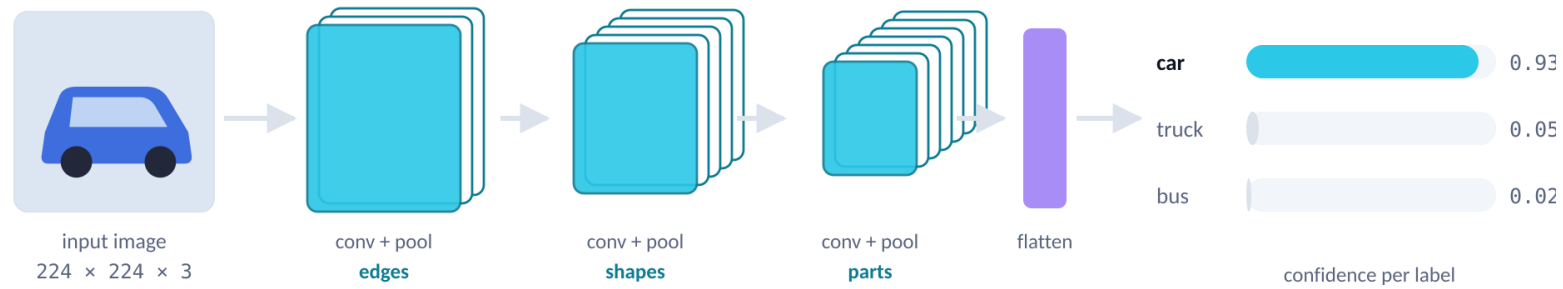
HOW A DETECTOR THINKS

- 1 Divide the image into a grid of regions.
- 2 For each region, predict: is there an object, where are its edges, and what is it?
- 3 Every guess carries a confidence score.
- 4 Overlapping boxes for the same object are merged, keeping the most confident one.



YOLO does all of this in a single pass — which is why it runs in real time.

Image Classification



- Assigning a label to an image
- Example: Cat vs. Dog
- Uses Convolutional Neural Networks (CNNs)

FROM PIXELS TO A LABEL

- 1 The network shrinks the image while deepening what it knows about it.
- 2 The last layer outputs one score per possible label.
- 3 Scores are normalised into confidences that add up to 100%.

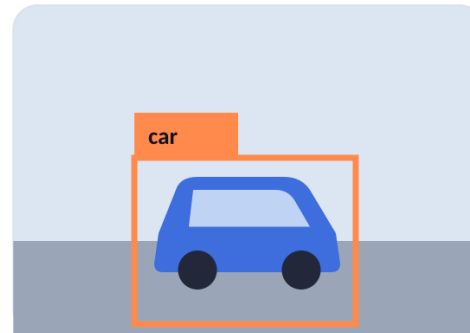
Semantic Segmentation

Classification



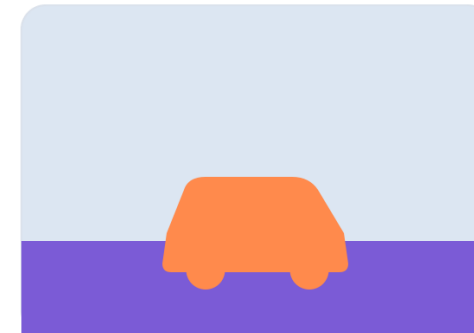
one label for the whole image

Detection



label + box for each object

Segmentation



a label for every single pixel

●lassifying each pixel in an image

●creates detailed maps of objects

●seful in medical imaging, self-driving

HOW PIXEL LABELLING WORKS

- 1 Compress the image down to understand what is present.
- 2 Expand it back to full size, assigning a class to every pixel.
- 3 The output is a mask, not a box — precise enough to outline a tumour.

Convolutional Neural Networks (CNNs)

INPUT PIXELS

10	12	14	90	180	200
11	13	40	120	190	205
12	16	60	150	195	210
13	18	70	160	200	215
14	20	80	170	205	220
15	22	90	175	210	225

FILTER (3×3)

-1	-1	-1
-1	8	-1
-1	-1	-1

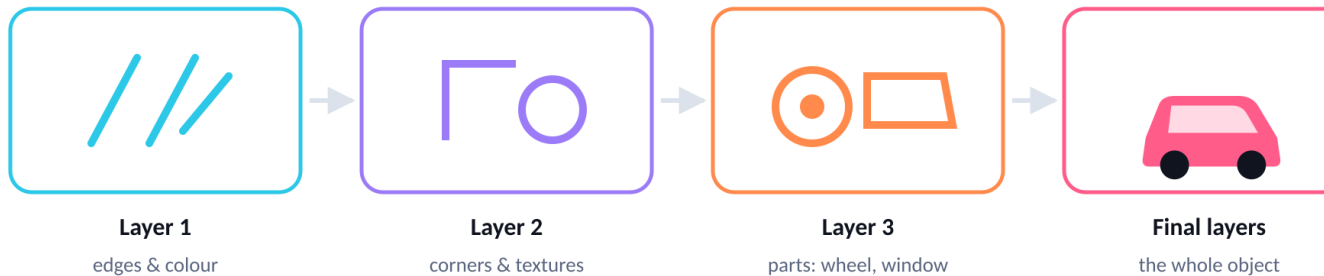
slides across the image

OUTPUT (FEATURE MAP)

0	42	96
0	58	110
0	66	118

Multiply each pixel by the filter value on top of it, add the nine results → one number.

One filter, one pass, one feature map. Stack thousands of these and you have a CNN.



Each layer combines the layer below it — simple patterns build into whole objects.

Backbone of modern vision systems

Convolution layers capture patterns

Examples: LeNet, ResNet

THE LEGO PRINCIPLE

- 1 Early layers learn tiny patterns: edges, corners, colour blobs.
- 2 Middle layers combine those into parts — a wheel, an eye, a window.
- 3 Late layers assemble parts into whole objects.
- 4 Nobody programs the filters; they are learned from examples.

Real-World Use Case – Healthcare



Analyzing X-rays and MRIs

Reading scans pixel by pixel, at volume



Early disease detection

Spotting pneumonia or lung cancer sooner



Faster diagnosis support

Assisting radiologists, not replacing them

WHY IT WORKS HERE

- 1 Medical images are consistent — same machine, same angle, same framing.
- 2 Labelled archives already exist from decades of diagnoses.
- 3 The model flags candidates; a clinician makes the call.
- 4 Value is in triage and speed, not in replacing judgement.

Real-World Use Case – Autonomous Vehicles



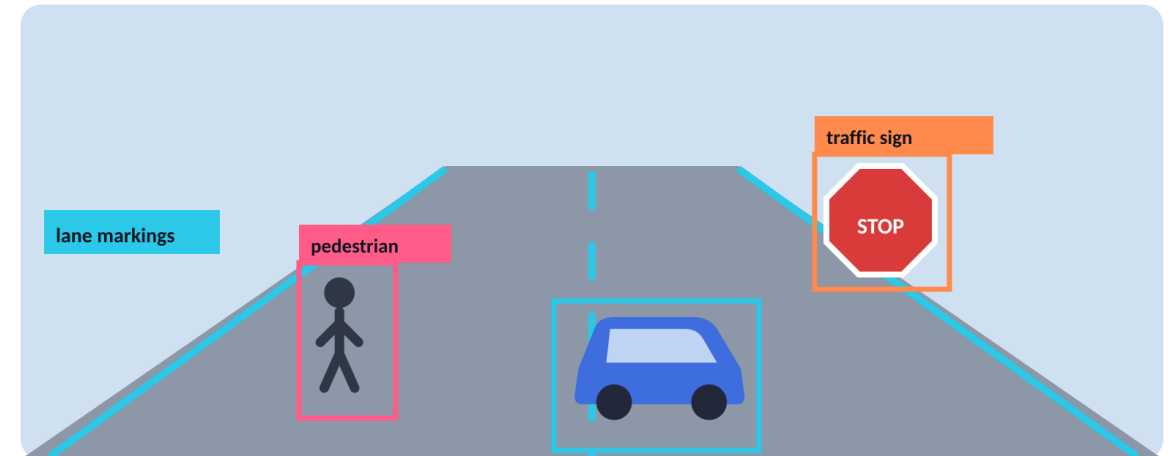
Lane detection



Pedestrian recognition



Traffic sign detection



THE REAL CONSTRAINT IS TIME

- 1 Dozens of frames per second must be processed, every one of them.
- 2 A detection that arrives 200 ms late is not a detection — which is why these models trade a little accuracy for speed.

Real-World Use Case – Retail



Smart checkout (Amazon Go)

Walk in, take what you need, walk out



Inventory monitoring

Shelves counted continuously instead of weekly

Detection + tracking + inventory

Three CV systems running together in real time, per store.



Customer behavior analysis

Which displays actually stop people

WHAT MAKES CHECKOUT-FREE HARD

- 1 Detection alone is not enough — the system must track one shopper across many cameras.
- 2 It has to notice an item leaving a shelf, and notice it being put back.
- 3 Hands, bags and other shoppers block the view constantly.
- 4 Every decision has to be right, because it becomes a charge on a card.

Real-World Use Case – Security



Facial recognition

Matching faces against a database, e.g. passport control



Surveillance systems

Continuous monitoring without continuous staffing



Anomaly detection

Flagging what does not fit the usual pattern

HOW FACE MATCHING WORKS

- 1 A face is converted into a list of numbers — a signature.
- 2 Two faces match when their signatures sit close together.
- 3 The system never compares photographs directly.

The flip side

This is the use case that raises the sharpest privacy questions. Capability is not the same as permission — ethical guidelines matter most exactly here.

Real-World Use Case – Agriculture



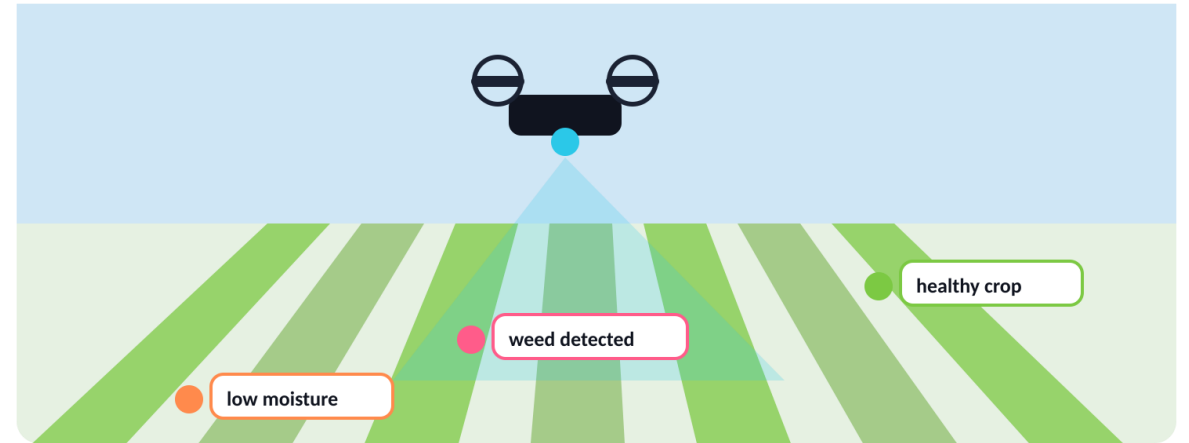
Crop monitoring via drones



Weed and pest detection



Yield prediction



WHY FARMERS CARE

- 1 Spraying a whole field costs the same whether or not the whole field needs it.
- 2 Spotting which square metres need water, fertiliser or pesticide turns a blanket cost into a targeted one — cheaper, and better for the land.

Real-World Use Case – Sports



Player tracking

Position and speed, every frame



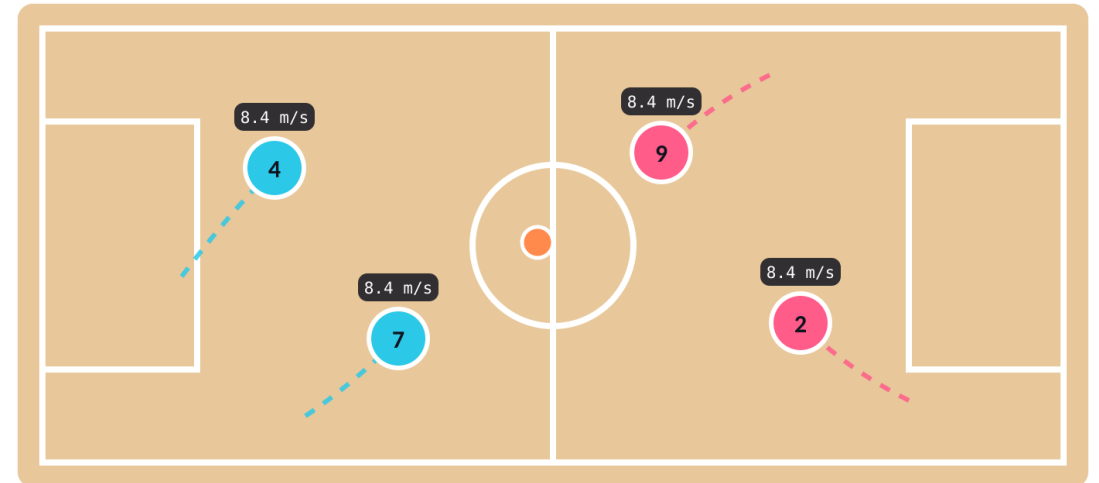
Performance analytics

Movement patterns that inform coaching



Automated highlights

Key moments detected and clipped automatically

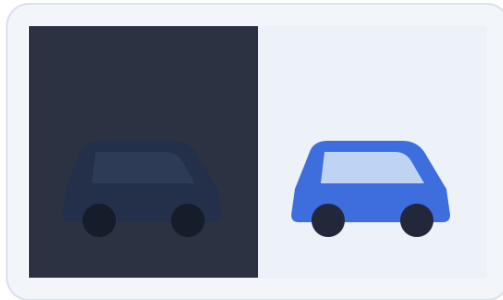


DETECTION VS TRACKING

- 1 Detection answers "who is on screen right now".
- 2 Tracking keeps the same identity attached to a player across frames — that is what makes the trail line possible.

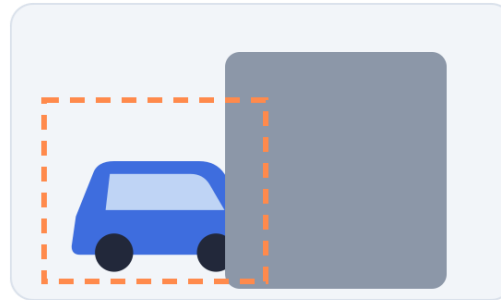
Challenges in Computer Vision

LIGHTING



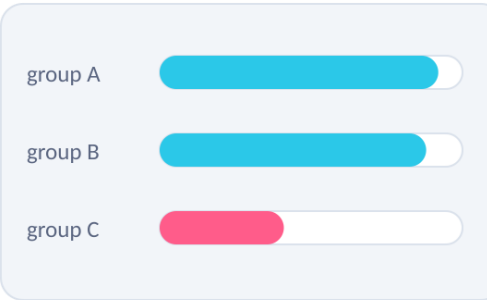
same car, two exposures

OCCLUSION



half the object is hidden

DATA & BIAS



accuracy follows the training data



Variations in lighting and angle



Occlusion (objects blocking each other)



Need for large datasets



Bias in training data

Facial recognition has performed measurably worse on darker skin tones — a data problem with real consequences.

Tools & Frameworks



OpenCV

Classical image processing — filters, edges, transforms



TensorFlow / Keras

Deep learning, production deployment



PyTorch

Deep learning, favoured for research



YOLOv8, Detectron2

Ready-made real-time detection and segmentation

WHICH ONE, WHEN

- 1 Cleaning up or transforming images, no learning involved → OpenCV.
- 2 Training your own model → PyTorch or TensorFlow. Detecting common objects today, with no training → YOLO or Detectron2.

Future of Computer Vision



Augmented Reality (AR)

Glasses that identify what you are looking at, live



Real-time video understanding

Following events over time, not just single frames



Integration with robotics and IoT

Machines that understand the room they are in



Generative AI for vision

Systems that both see and create images

The direction of travel: from recognising objects to understanding scenes.

Key Takeaways



CV = teaching computers to see and understand

Pixels in, meaning out



Core tasks: classification, detection, segmentation

A label, a box, or every pixel



Real-world impact: healthcare, cars, retail, agriculture

Already deployed, not theoretical



Future: AR, robotics, generative AI

Increasingly part of everyday life

Thank you